INFSCI 1540 - Data Engineering
Climate Data Warehouse Project
Final Report

Boyi Sun Murphy Zhao

April 24, 2026

1. Project Overview

This project builds a climate data warehouse based on NOAA Climate Data Online data. The whole project can be founded in It uses Docker, Kafka, ksqlDB, MySQL, and Python to support data ingestion, cleaning, transformation, and storage. The system organizes climate normal data in a STAR schema with fact and dimension tables, and also creates pre-aggregated monthly summary tables for efficient analysis. By integrating multiple weather stations into the pipeline, the project supports comparative climate analysis across locations and time periods. All project materials are available in the project GitHub repository: https://github.com/Boyiiiisun/INFSCI1540-Project-Climate.

2. Data Description

This project maintains climate normal data derived from the NOAA Climate Data Online source. The data source is available at https://www.ncei.noaa.gov/cdo-web/, and the implementation in the current repository is based on the `NORMAL_DLY` sample file, which stores daily normal measurements for multiple weather stations. The data pipeline currently preserves four stations in the repository sample: *PETERSBURG 2 N ND US*, *PITTSBURGH INTL AP PA US*, *SEATTLE URBAN SITE WA US*, and *SAN FRANCISCO OCEANSD CA US*. Each record is associated with one station and one calendar date.

The source file contains the following attributes: station identifier, station name, elevation, latitude, longitude, date, daily minimum temperature normal, daily maximum temperature normal, and month-to-date precipitation normal. The system preserves station metadata together with daily climate observations so that the data warehouse can support time-based analysis, station-based comparison, and pre-aggregated monthly summaries.

The grain of the main fact data in the warehouse is one climate normal observation for one station on one date. This design allows the system to support both daily-level analysis and month-level summaries derived from the daily observations.

3. Docker Containers Used in the System and Corresponding `docker-compose.yml`

The system is deployed through Docker Compose. The repository uses containerized infrastructure to isolate operational data storage, warehouse storage, streaming middleware, and query interfaces. The main containers are summarized below.

3.1 Container Roles

- **web-server**: a lightweight Apache/PHP container used as a simple web server.
- **mysql-odb-server**: the MySQL instance that stores the operational database (ODB) raw table.
- **mysql-dw-server**: the MySQL instance that stores the dimensional warehouse tables and summary tables.
- **phpmyadmin-odb**: a browser-based administration interface for the ODB MySQL instance.
- **phpmyadmin-dw**: a browser-based administration interface for the DW MySQL instance.
- **zookeeper**: the coordination service required by Kafka in this environment.
- **broker**: the Kafka broker that receives raw JSON messages from the producer and exposes them to downstream consumers and ksqlDB.
- **ksqldb-server**: the ksqlDB server used for stream definition, cleaning, transformation, and monthly aggregation.
- **ksqldb-cli**: the command-line client used to execute ksql statements interactively.

This container layout separates storage and streaming concerns. The ODB is used as a staging layer for cleaned operational records, while the DW stores the star schema and summary tables. Kafka and ksqlDB provide the streaming and transformation layer between ingestion and warehouse analytics.

After running `docker-compose up -d` in the terminal, the Docker environment starts all required services for the project. The screenshot below shows the running containers, confirming that the ODB, DW, Kafka, ksqlDB, phpMyAdmin, and web-server components were successfully launched before the data preparation and warehouse loading steps.

```
PS C:\Users\孙博弈\Desktop\INFSCI 1540\Project\INFSCI1540-Project-Climate\climate> docker-compose up -d
[+] up 10/10
 ✓ Network climate_default              Created                                    0.1s
 ✓ Container climate-mysql-odb-server-1 Created                                    0.3s
 ✓ Container climate-web-server-1       Created                                    0.3s
 ✓ Container climate-mysql-dw-server-1  Created                                    0.3s
 ✓ Container zookeeper                  Created                                    0.3s
 ✓ Container climate-phpmyadmin-odb-1   Created                                    0.2s
 ✓ Container climate-phpmyadmin-dw-1    Created                                    0.2s
 ✓ Container broker                     Created                                    0.2s
 ✓ Container ksqldb-server              Created                                    0.1s
 ✓ Container ksqldb-cli                 Created                                    0.1s
```

3.2 Docker Compose File

```
#version: '2'

services:
  web-server:
    image: php:7.4.3-apache
    volumes:
      - "./html/:/var/www/html/"
    ports:
      - "8080:80"

  mysql-odb-server:
    image: mysql:8.0.23
    platform: linux/amd64
    environment:
      MYSQL_ROOT_PASSWORD: secret
    volumes:
      - mysql-odb-data:/var/lib/mysql
    ports:
      - "13306:3306"

  mysql-dw-server:
    image: mysql:8.0.23
    platform: linux/amd64
    environment:
      MYSQL_ROOT_PASSWORD: secret
    volumes:
      - mysql-dw-data:/var/lib/mysql
    ports:
      - "23306:3306"

  phpmyadmin-odb:
    image: phpmyadmin/phpmyadmin:5.0.1
    platform: linux/amd64
    environment:
      PMA_HOST: mysql-odb-server
      PMA_USER: root
      PMA_PASSWORD: secret
    ports:
      - "15000:80"
    depends_on:
      - mysql-odb-server

  phpmyadmin-dw:
```

```
    image: phpmyadmin/phpmyadmin:5.0.1
    platform: linux/amd64
    environment:
      PMA_HOST: mysql-dw-server
      PMA_USER: root
      PMA_PASSWORD: secret
    ports:
      - "25000:80"
    depends_on:
      - mysql-dw-server

  zookeeper:
    image: confluentinc/cp-zookeeper:7.9.4
    platform: linux/amd64
    hostname: zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  broker:
    image: confluentinc/cp-enterprise-kafka:7.9.4
    platform: linux/amd64
    hostname: broker
    container_name: broker
    depends_on:
      - zookeeper
    ports:
      - "29092:29092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://broker:9092,PLAINTEXT_HOST://localhost
    :29092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1

  ksqldb-server:
    image: confluentinc/cp-ksqldb-server:7.9.6-1-ubi8
```

```
    platform: linux/amd64
    hostname: ksqldb-server
    container_name: ksqldb-server
    depends_on:
      - broker
    ports:
      - "8088:8088"
    environment:
      KSQL_LISTENERS: http://0.0.0.0:8088
      KSQL_BOOTSTRAP_SERVERS: broker:9092
      KSQL_KSQL_LOGGING_PROCESSING_STREAM_AUTO_CREATE: "true"
      KSQL_KSQL_LOGGING_PROCESSING_TOPIC_AUTO_CREATE: "true"
      CONFLUENT_SUPPORT_METRICS_ENABLE: "false"

  ksqldb-cli:
    image: confluentinc/cp-ksqldb-cli:7.9.6-1-ubi8
    platform: linux/amd64
    container_name: ksqldb-cli
    depends_on:
      - broker
      - ksqldb-server
    entrypoint: /bin/sh
    tty: true

volumes:
  mysql-odb-data:
  mysql-dw-data:
```

4. Data Preparation Process, Cleaning, and Transformation

The data preparation process is implemented as a multi-stage pipeline.

4.1 Step 1: CSV Ingestion into Kafka

The script `dly_producer.py` reads the source CSV file and publishes one JSON message per record to the Kafka topic `NORMAL_DLY_RAW`. During this step, the script performs basic type conversion. Numerical values such as elevation, latitude, longitude, and the three climate measures are converted from text into numeric Python values before the record is published.

4.2 Step 2: Loading Raw Operational Data into ODB

The script `consumer.py` consumes messages from `NORMAL_DLY_RAW` and writes them into the operational database table `climate_normal_daily_raw`. In this stage, the process performs the following cleaning operations:

- parsing the date field from `YYYYMMDD` into SQL `DATE` format,
- converting numeric values to decimal types,
- skipping malformed messages when required attributes are missing,
- using `ON DUPLICATE KEY UPDATE` to avoid duplicate station-date records.

The ODB serves as a structured staging area between raw stream ingestion and dimensional loading.

4.3 Step 3: ksqlDB Stream Cleaning and Transformation

The repository includes a ksqlDB layer that transforms the raw Kafka topic into analysis-ready streams. First, the raw JSON topic is registered as the ksql stream `normal_dly_raw`. Then the file `create_clean_stream.sql` derives additional time attributes from the raw date string and casts the climate fields into doubles.

```
CREATE STREAM normal_dly_clean AS
SELECT
  station_id,
  station_name,
  elevation,
  latitude,
  longitude,
  norm_date,
  CAST(SUBSTRING(norm_date, 1, 4) AS INTEGER) AS year_num,
  CAST(SUBSTRING(norm_date, 5, 2) AS INTEGER) AS month_num,
  SUBSTRING(norm_date, 1, 6) AS norm_month,
  CAST(dly_tmin_normal AS DOUBLE) AS dly_tmin_normal,
  CAST(dly_tmax_normal AS DOUBLE) AS dly_tmax_normal,
  CAST(mtd_prcp_normal AS DOUBLE) AS mtd_prcp_normal
FROM normal_dly_raw
EMIT CHANGES;
```

This transformation is important because it introduces month-level and year-level attributes that are later used for summary aggregation.

4.4 Step 4: Loading the Star Schema

The script `load.py` reads the cleaned operational data from ODB and populates the warehouse star schema. The process:

- extracts unique stations and upserts them into `dim_station`,
- extracts unique dates and inserts them into `dim_date`,

- maps station identifiers to surrogate keys,
- loads the fact table `fact_daily_climate_normal`.

This loading step also uses idempotent insertion logic so the process can be rerun without creating duplicate warehouse records.

5. STAR Schema Design

The data warehouse uses a classical star schema with one fact table and two dimension tables.

5.1 Schema Overview

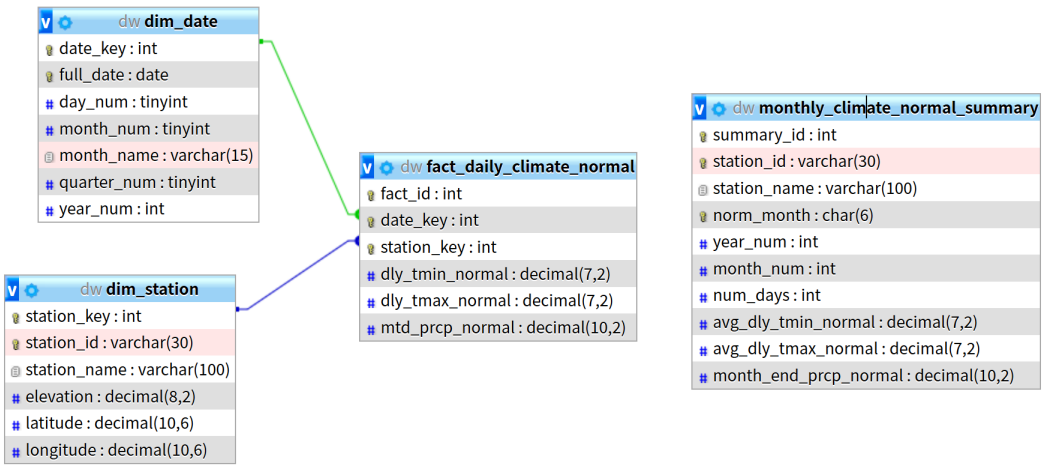

Figure 1: STAR schema design for the climate data warehouse.

The grain of the fact table is one climate normal record for one station on one date.

5.2 Dimension Table: `dim_date`

```sql
CREATE TABLE IF NOT EXISTS dim_date (
    date_key INT NOT NULL PRIMARY KEY,
    full_date DATE NOT NULL,
    day_num TINYINT NOT NULL,
    month_num TINYINT NOT NULL,
    month_name VARCHAR(15) NOT NULL,
    quarter_num TINYINT NOT NULL,
    year_num INT NOT NULL,
    CONSTRAINT uq_dim_date_full_date UNIQUE (full_date)
);
```

5.3 Dimension Table: `dim_station`

```sql
CREATE TABLE IF NOT EXISTS dim_station (
    station_key INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    station_id VARCHAR(30) NOT NULL,
    station_name VARCHAR(100) NOT NULL,
    elevation DECIMAL(8,2) NULL,
    latitude DECIMAL(10,6) NULL,
    longitude DECIMAL(10,6) NULL,
    CONSTRAINT uq_dim_station_station_id UNIQUE (station_id)
);
```

5.4 Fact Table: `fact_daily_climate_normal`

```sql
CREATE TABLE IF NOT EXISTS fact_daily_climate_normal (
    fact_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    date_key INT NOT NULL,
    station_key INT NOT NULL,
    dly_tmin_normal DECIMAL(7,2) NULL,
    dly_tmax_normal DECIMAL(7,2) NULL,
    mtd_prcp_normal DECIMAL(10,2) NULL,
    CONSTRAINT uq_fact_station_date UNIQUE (station_key, date_key),
    CONSTRAINT fk_fact_daily_normal_date
        FOREIGN KEY (date_key) REFERENCES dim_date(date_key),
    CONSTRAINT fk_fact_daily_normal_station
        FOREIGN KEY (station_key) REFERENCES dim_station(station_key)
);
```

5.5 SQL Statements Used to Populate the STAR Schema

The warehouse is populated through SQL statements executed by `load.py`. The main statements are listed below.

Populate `dim_station`

```sql
INSERT INTO dim_station (
    station_id, station_name, elevation, latitude, longitude
) VALUES (%s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    station_name = VALUES(station_name),
    elevation = VALUES(elevation),
    latitude = VALUES(latitude),
    longitude = VALUES(longitude);
```

Populate dim_date

```
INSERT IGNORE INTO dim_date (
    date_key, full_date, day_num, month_num, month_name, quarter_num, year_num
) VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Populate fact_daily_climate_normal

```
INSERT INTO fact_daily_climate_normal (
    date_key, station_key, dly_tmin_normal, dly_tmax_normal, mtd_prcp_normal
) VALUES (%s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    dly_tmin_normal = VALUES(dly_tmin_normal),
    dly_tmax_normal = VALUES(dly_tmax_normal),
    mtd_prcp_normal = VALUES(mtd_prcp_normal);
```

6. Data Streams

Following the data warehousing workflow introduced in Lab 7 and the ksqlDB-based stream processing approach in Lab 9, our project uses Kafka and ksqlDB to support real-time ingestion and aggregation of climate data before loading the processed results into the final data warehouse.

6.1 Overview of the Stream Pipeline

The data stream pipeline in our project consists of four main stages:

1. Climate observation records are read from the prepared source file.
2. A Python producer sends these records to a Kafka topic.
3. ksqlDB defines the Kafka topic as a structured stream and performs continuous processing and aggregation.
4. The aggregated results are loaded into the MySQL data warehouse for multidimensional analysis and OLAP queries.

Compared with the original Lab 7 workflow, our project replaces the intermediate `odb.transaction` layer with a ksqlDB stream-processing layer. This allows the system to process incoming events directly from Kafka and generate aggregated results in real time.

6.2 Source Data Stream

The source data stream contains cleaned climate observation records. Each record represents one weather observation event and includes attributes such as observation date, station identifier, location, measurement type, and observed value.

These records are produced by a Python data producer and sent to a Kafka topic. In our project, this topic serves as the entry point of the streaming pipeline.

6.3 Kafka Topic and Stream Definition

After the producer sends the climate records to Kafka, ksqlDB is used to associate a schema with the underlying topic. In this way, raw Kafka messages are interpreted as structured stream records.

The stream definition logically represents the incoming climate events and acts as the streaming equivalent of the original transaction-level input layer. Instead of inserting incoming records into an ODB transaction table first, our project directly defines them as a ksqlDB stream.

6.4 Continuous Query Processing

To monitor incoming events, we use a continuous query over the climate event stream. This query continuously outputs new rows as data arrives in the Kafka topic.

This mechanism is useful for validating that the data producer is sending records correctly and that the stream pipeline is functioning as expected. It also demonstrates the real-time nature of the system, since newly produced climate observations immediately become visible in the stream query results.

6.5 Continuous Aggregation with ksqlDB

After the climate event stream is created, ksqlDB is used to define a continuous aggregation query. This query groups incoming records by selected attributes and incrementally computes aggregate values as new events arrive.

For our climate data warehouse, this aggregation step can be used to compute summarized measures such as monthly average temperature, monthly maximum temperature, monthly minimum temperature, and monthly total precipitation by station.

The aggregated output is maintained as a ksqlDB table. This table replaces the aggregation logic that was originally implemented in Lab 7 through the `odb_producer.py` script and the SQL query over the `transaction` table.

6.6 Loading Aggregated Results into the Data Warehouse

The aggregated results produced by ksqlDB are then loaded into the MySQL data warehouse. These results populate the fact table and support the creation of pre-aggregated summary tables.

The MySQL data warehouse remains the final storage layer of the project. It is used to maintain the STAR schema, support joins with dimension tables, and execute OLAP queries for climate trend analysis.

6.7 Benefits of the Modified Stream Design

This modified data stream design provides several advantages:

- It simplifies the original Lab 7 pipeline by removing the intermediate ODB transaction storage layer.

- It incorporates real-time stream processing using ksqlDB.
- It supports continuous aggregation directly on top of Kafka streams.
- It better demonstrates the integration of Kafka, ksqlDB, and MySQL in a modern data engineering workflow.

Overall, the modified stream architecture allows our project to combine the data warehousing logic of Lab 7 with the stream-processing capabilities of Lab 9, resulting in a more efficient and more scalable climate data pipeline.

7. Specification of Pre-Aggregated Summary Tables

The project includes a monthly pre-aggregated summary table in the warehouse and a corresponding aggregated table in ksqlDB.

7.1 Warehouse Summary Table DDL

```
CREATE TABLE IF NOT EXISTS monthly_climate_normal_summary (
    summary_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    station_id VARCHAR(30) NOT NULL,
    station_name VARCHAR(100) NOT NULL,
    norm_month CHAR(6) NOT NULL,
    year_num INT NOT NULL,
    month_num INT NOT NULL,
    num_days INT NULL,
    avg_dly_tmin_normal DECIMAL(7,2) NULL,
    avg_dly_tmax_normal DECIMAL(7,2) NULL,
    month_end_prcp_normal DECIMAL(10,2) NULL,
    CONSTRAINT uq_monthly_normal_station_month UNIQUE (station_id, norm_month)
);
```

7.2 SQL for Creation and Population of Summary Data in ksqlDB

```
CREATE TABLE normal_dly_monthly_summary
WITH (
  KEY_FORMAT = 'JSON',
  VALUE_FORMAT = 'JSON'
) AS
SELECT
  station_id,
  AS_VALUE(station_id) AS station_id_v,
  station_name,
  AS_VALUE(station_name) AS station_name_v,
```

```
  norm_month,
  AS_VALUE(norm_month) AS norm_month_v,
  year_num,
  AS_VALUE(year_num) AS year_num_v,
  month_num,
  AS_VALUE(month_num) AS month_num_v,
  COUNT(*) AS num_days,
  AVG(dly_tmin_normal) AS avg_dly_tmin_normal,
  AVG(dly_tmax_normal) AS avg_dly_tmax_normal,
  MAX(mtd_prcp_normal) AS month_end_prcp_normal
FROM normal_dly_clean
GROUP BY station_id, station_name, norm_month, year_num, month_num
EMIT CHANGES;
```

7.3 Loading Summary Data into the Warehouse

The script `load_monthly_summary.py` consumes the summary topic and upserts the aggregated rows into the warehouse summary table using the following SQL pattern.

```
INSERT INTO monthly_climate_normal_summary (
    station_id,
    station_name,
    norm_month,
    year_num,
    month_num,
    num_days,
    avg_dly_tmin_normal,
    avg_dly_tmax_normal,
    month_end_prcp_normal
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
    station_name = VALUES(station_name),
    year_num = VALUES(year_num),
    month_num = VALUES(month_num),
    num_days = VALUES(num_days),
    avg_dly_tmin_normal = VALUES(avg_dly_tmin_normal),
    avg_dly_tmax_normal = VALUES(avg_dly_tmax_normal),
    month_end_prcp_normal = VALUES(month_end_prcp_normal);
```

8. OLAP Data Warehouse Queries

This section presents five OLAP queries executed against the climate data warehouse. Each query uses the warehouse fact table together with the date and station dimensions to support station comparison, temporal aggregation, and climate pattern analysis.

Before running the OLAP queries, the raw NOAA-style measures are converted into easier-to-read imperial units. In the warehouse, daily minimum and maximum temperature are stored in tenths of degrees Fahrenheit, and month-to-date precipitation is stored in hundredths of inches. To make the analytical results more interpretable, the OLAP workflow first converts temperature values by dividing by 10 and precipitation values by dividing by 100. These converted values are then reused through shared views so that all five queries use the same unit definitions and monthly aggregation logic.

8.1 Shared Unit Conversion and Monthly Aggregation Views

```sql
USE dw;

-- Convert raw NOAA-style units once:
--   temperature: stored in tenths of degrees Fahrenheit
--   precipitation: stored in hundredths of inches
CREATE OR REPLACE VIEW vw_daily_climate_imperial AS
SELECT
    ds.station_id,
    ds.station_name,
    ds.latitude,
    ds.longitude,
    dd.year_num,
    dd.month_num,
    dd.month_name,
    dd.quarter_num,
    dd.full_date,
    ROUND(f.dly_tmin_normal / 10.0, 2) AS dly_tmin_normal_f,
    ROUND(f.dly_tmax_normal / 10.0, 2) AS dly_tmax_normal_f,
    ROUND(f.mtd_prcp_normal / 100.0, 2) AS mtd_prcp_normal_in
FROM fact_daily_climate_normal f
JOIN dim_date dd
    ON f.date_key = dd.date_key
JOIN dim_station ds
    ON f.station_key = ds.station_key
WHERE
    f.dly_tmin_normal IS NOT NULL
    AND f.dly_tmax_normal IS NOT NULL
    AND f.mtd_prcp_normal IS NOT NULL;

CREATE OR REPLACE VIEW vw_monthly_climate_imperial AS
SELECT
    station_id,
    station_name,
    latitude,
```

```
    longitude,
    year_num,
    month_num,
    month_name,
    COUNT(*) AS num_days,
    ROUND(AVG((dly_tmin_normal_f + dly_tmax_normal_f) / 2), 2)
        AS monthly_avg_temperature_f,
    ROUND(MAX(dly_tmax_normal_f), 2) AS monthly_max_temperature_f,
    ROUND(MIN(dly_tmin_normal_f), 2) AS monthly_min_temperature_f,
    ROUND(MAX(mtd_prcp_normal_in), 2) AS monthly_total_precipitation_in
FROM vw_daily_climate_imperial
GROUP BY
    station_id,
    station_name,
    latitude,
    longitude,
    year_num,
    month_num,
    month_name;
```

These two views standardize the OLAP input. The first view performs the unit conversion at the daily level, while the second view rolls the converted values into monthly station summaries. All five OLAP queries below read from these views instead of repeating the same conversion and aggregation logic.

8.2 Query 1: Monthly Climate Summary by Station

```
-- OLAP Query 1:
-- Monthly station-level climate summary using converted units.
SELECT
    station_id,
    station_name,
    year_num,
    month_num,
    month_name,
    num_days,
    monthly_avg_temperature_f AS monthly_avg_temperature_normal,
    monthly_max_temperature_f AS monthly_max_temperature_normal,
    monthly_min_temperature_f AS monthly_min_temperature_normal,
    monthly_total_precipitation_in AS monthly_total_precipitation_normal
FROM vw_monthly_climate_imperial
ORDER BY
    station_name,
    year_num,
```

```
    month_num;
```

Showing rows 0 - ... (Query took 0.0080 seconds.) [station_name: PETERSBURG 2 N ND US... - SAN FRANCISCO OCEANSD CA US...] [year_num: 2010... - 2010...] [month_num: 1... - 1...]

```
-- OLAP Query 1: -- Monthly station-level climate summary using converted units. SELECT station_id, station_name, year_num, month_num, month_name, num_days, monthly_avg_temperature_f AS monthly_avg_temperature_normal, month
monthly_min_temperature_f AS monthly_min_temperature_normal, monthly_total_precipitation_in AS monthly_total_precipitation_normal FROM vw_monthly_climate_imperial ORDER BY station_name, year_num, month_num
```

> >> Show all | Number of rows: 25 Filter rows: Search this table

+ Options

station_id	station_name	year_num	month_num	month_name	num_days	monthly_avg_temperature_normal	monthly_max_temperature_normal	monthly_min_temperature_normal	monthly_total_precipitation_normal
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	1	January	31	5.10	16.20	-4.80	0.55
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	2	February	28	10.15	23.90	-3.50	0.44
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	3	March	31	22.90	41.20	6.10	0.72
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	4	April	30	40.55	60.00	22.60	0.98
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	5	May	31	53.64	70.20	36.00	2.49
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	6	June	30	63.21	77.40	47.70	3.76
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	7	July	31	67.85	80.50	55.50	3.22
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	8	August	31	66.54	80.50	50.40	2.75
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	9	September	30	56.15	74.90	37.40	2.08
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	10	October	31	42.05	61.20	24.80	1.69
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	11	November	30	25.10	43.30	8.40	0.81
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	12	December	31	10.10	24.40	-3.10	0.73
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	1	January	31	28.40	36.50	20.90	2.70
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	2	February	28	31.15	42.90	21.40	2.39
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	3	March	31	39.60	55.60	25.50	2.95
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	4	April	30	50.95	66.60	35.40	3.11
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	5	May	31	60.05	74.90	44.90	3.95
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	6	June	30	68.75	82.10	54.20	4.30
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	7	July	31	72.66	82.70	61.90	3.83
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	8	August	31	71.45	82.20	59.20	3.48
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	9	September	30	64.15	79.40	48.30	3.11
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	10	October	31	52.75	68.00	38.80	2.29
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	11	November	30	42.95	57.20	30.20	3.23
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	12	December	31	32.35	44.30	21.90	2.85
GHCND:USC00047767	SAN FRANCISCO OCEANSD CA US	2010	1	January	31	50.80	57.70	44.70	4.18

> >> Show all | Number of rows: 25 Filter rows: Search this table

Figure 2: Result of Query 1: monthly climate summary by station.

Query 1 reads directly from the shared monthly view and reports the number of observations, monthly average temperature, monthly maximum temperature, monthly minimum temperature, and monthly precipitation normal for each station. The result is meaningful because it presents monthly summaries in already-converted imperial units, making it easier to compare seasonal climate patterns across all stations.

8.3 Query 2: Coastal Seattle vs. Inland Petersburg Monthly Comparison

```
-- OLAP Query 2:
-- Compare similar-latitude coastal and inland stations:
-- Seattle Urban Site, WA vs Petersburg 2 N, ND.
WITH comparison_months AS (
    SELECT
        year_num,
        month_num,
        month_name,
        MAX(CASE
            WHEN station_id = 'GHCND:USW00024281'
            THEN monthly_avg_temperature_f
        END) AS seattle_avg_temperature,
        MAX(CASE
            WHEN station_id = 'GHCND:USC00327027'
            THEN monthly_avg_temperature_f
        END) AS petersburg_avg_temperature,
```

```sql
        MAX(CASE
            WHEN station_id = 'GHCND:USW00024281'
            THEN monthly_total_precipitation_in
        END) AS seattle_total_precipitation,
        MAX(CASE
            WHEN station_id = 'GHCND:USC00327027'
            THEN monthly_total_precipitation_in
        END) AS petersburg_total_precipitation
    FROM vw_monthly_climate_imperial
    WHERE station_id IN ('GHCND:USW00024281', 'GHCND:USC00327027')
    GROUP BY
        year_num,
        month_num,
        month_name
)
SELECT
    year_num,
    month_num,
    month_name,
    seattle_avg_temperature,
    petersburg_avg_temperature,
    ROUND(
        seattle_avg_temperature - petersburg_avg_temperature,
        2
    ) AS seattle_minus_petersburg_avg_temperature,
    seattle_total_precipitation,
    petersburg_total_precipitation,
    ROUND(
        seattle_total_precipitation - petersburg_total_precipitation,
        2
    ) AS seattle_minus_petersburg_precipitation
FROM comparison_months
ORDER BY
    year_num,
    month_num;
```

Query 2 compares Seattle and Petersburg by pivoting both stations into the same monthly row from the shared monthly view. This layout makes the temperature and precipitation differences directly visible for each month. The result is useful because it highlights the contrast between a coastal station and an inland station, showing how geography can influence both temperature and precipitation normals.

8.4 Query 3: Seasonal Comparison Between San Francisco and Pittsburgh

Showing rows 0 - 11 (12 total, Query took 0.0082 seconds.)

```
-- OLAP Query 2: -- Compare similar-latitude coastal and inland stations: -- Seattle Urban Site, WA vs Petersburg 2 N, ND. WITH comparison_months AS ( SELECT year_num, month_num, month_name, MAX(CASE WHEN st
seattle_avg_temperature, MAX(CASE WHEN station_id = 'GHCND:USC00327027' THEN monthly_avg_temperature_f END) AS petersburg_avg_temperature, MAX(CASE WHEN station_id = 'GHCND:USW00024281' THEN monthly_total_p
'GHCND:USC00327027' THEN monthly_total_precipitation_in END) AS petersburg_total_precipitation FROM vw_monthly_climate_imperial WHERE station_id IN ('GHCND:USW00024281', 'GHCND:USC00327027') GROUP BY ye|...
```

+ Options

year_num	month_num	month_name	seattle_avg_temperature	petersburg_avg_temperature	seattle_minus_petersburg_avg_temperature	seattle_total_precipitation	petersburg_total_precipitation	seattle_minus_petersburg_precipitation
2010	1	January	42.96	5.10	37.86	5.47	0.55	4.92
2010	2	February	44.26	10.15	34.11	3.52	0.44	3.08
2010	3	March	47.55	22.90	24.65	3.85	0.72	3.13
2010	4	April	51.41	40.55	10.86	2.79	0.98	1.81
2010	5	May	57.30	53.64	3.66	2.01	2.49	-0.48
2010	6	June	61.99	63.21	-1.22	1.57	3.76	-2.19
2010	7	July	66.15	67.85	-1.70	0.85	3.22	-2.37
2010	8	August	66.60	66.54	0.06	0.94	2.75	-1.81
2010	9	September	61.95	56.15	5.80	1.30	2.08	-0.78
2010	10	October	53.75	42.05	11.70	3.38	1.69	1.69
2010	11	November	46.50	25.10	21.40	5.98	0.81	5.17
2010	12	December	41.45	10.10	31.35	6.06	0.73	5.33

Figure 3: Result of Query 2: coastal Seattle versus inland Petersburg.

```sql
-- OLAP Query 3:
-- Compare San Francisco, CA vs Pittsburgh, PA by season.
WITH seasonal_station AS (
    SELECT
        station_id,
        station_name,
        year_num,
        CASE
            WHEN month_num IN (12, 1, 2) THEN 'Winter'
            WHEN month_num IN (3, 4, 5) THEN 'Spring'
            WHEN month_num IN (6, 7, 8) THEN 'Summer'
            ELSE 'Fall'
        END AS season_name,
        CASE
            WHEN month_num IN (12, 1, 2) THEN 1
            WHEN month_num IN (3, 4, 5) THEN 2
            WHEN month_num IN (6, 7, 8) THEN 3
            ELSE 4
        END AS season_order,
        ROUND(AVG(monthly_avg_temperature_f), 2) AS seasonal_avg_temperature,
        ROUND(MAX(monthly_max_temperature_f), 2) AS seasonal_max_temperature,
        ROUND(MIN(monthly_min_temperature_f), 2) AS seasonal_min_temperature,
        ROUND(SUM(monthly_total_precipitation_in), 2)
            AS seasonal_total_precipitation
    FROM vw_monthly_climate_imperial
    WHERE station_id IN ('GHCND:USC00047767', 'GHCND:USW00094823')
    GROUP BY
        station_id,
        station_name,
        year_num,
        season_name,
        season_order
```

```
)
SELECT
    year_num,
    season_name,
    station_name,
    seasonal_avg_temperature,
    seasonal_max_temperature,
    seasonal_min_temperature,
    ROUND(seasonal_max_temperature - seasonal_min_temperature, 2)
        AS seasonal_temperature_range,
    seasonal_total_precipitation
FROM seasonal_station
ORDER BY
    year_num,
    season_order,
    station_name;
```

Showing rows 0 - 7 (8 total, Query took 0.0058 seconds.)

```
-- OLAP Query 3: -- Compare San Francisco, CA vs Pittsburgh, PA by season. WITH seasonal_station AS ( SELECT station_id, station_name, year_num, CASE WHEN month_num IN (12, 1,
'Fall' END AS season_name, CASE WHEN month_num IN (12, 1, 2) THEN 1 WHEN month_num IN (3, 4, 5) THEN 2 WHEN month_num IN (6, 7, 8) THEN 3 ELSE 4 END AS season_order, ROUND(AVG(
seasonal_max_temperature, ROUND(MIN(monthly_min_temperature_f), 2) AS seasonal_min_temperature, ROUND(SUM(monthly_total_precipitation_in), 2) AS seasonal_total_precipitation FR
```

+ Options

year_num	season_name	station_name	seasonal_avg_temperature	seasonal_max_temperature	seasonal_min_temperature	seasonal_temperature_range	seasonal_total_precipitation
2010	Winter	PITTSBURGH INTL AP PA US	30.63	44.30	20.90	23.40	7.94
2010	Winter	SAN FRANCISCO OCEANSD CA US	51.47	59.10	44.70	14.40	12.25
2010	Spring	PITTSBURGH INTL AP PA US	50.20	74.90	25.50	49.40	10.01
2010	Spring	SAN FRANCISCO OCEANSD CA US	54.08	60.00	47.10	12.90	4.45
2010	Summer	PITTSBURGH INTL AP PA US	70.95	82.70	54.20	28.50	11.61
2010	Summer	SAN FRANCISCO OCEANSD CA US	57.82	63.50	51.60	11.90	0.27
2010	Fall	PITTSBURGH INTL AP PA US	53.28	79.40	30.20	49.20	8.63
2010	Fall	SAN FRANCISCO OCEANSD CA US	57.74	64.70	46.60	18.10	3.71

Figure 4: Result of Query 3: seasonal comparison between San Francisco and Pittsburgh.

Query 3 rolls monthly climate values from the shared view into seasonal summaries for San Francisco and Pittsburgh. It reports seasonal average temperature, maximum and minimum normal temperatures, seasonal temperature range, and total precipitation. The result is meaningful because it moves beyond monthly detail and shows broader regional differences, especially the contrast between San Francisco's milder coastal climate and Pittsburgh's stronger seasonal variation.

8.5 Query 4: Seattle vs. San Francisco Monthly Difference Ranking

```
-- OLAP Query 4:
-- Compare Seattle, WA vs San Francisco, CA and rank monthly temperature gaps.
WITH paired_months AS (
    SELECT
        year_num,
        month_num,
        month_name,
        MAX(CASE
```

```
            WHEN station_id = 'GHCND:USW00024281'
            THEN latitude
        END) AS seattle_latitude,
        MAX(CASE
            WHEN station_id = 'GHCND:USC00047767'
            THEN latitude
        END) AS san_francisco_latitude,
        MAX(CASE
            WHEN station_id = 'GHCND:USW00024281'
            THEN monthly_avg_temperature_f
        END) AS seattle_avg_temperature,
        MAX(CASE
            WHEN station_id = 'GHCND:USC00047767'
            THEN monthly_avg_temperature_f
        END) AS san_francisco_avg_temperature,
        MAX(CASE
            WHEN station_id = 'GHCND:USW00024281'
            THEN monthly_total_precipitation_in
        END) AS seattle_total_precipitation,
        MAX(CASE
            WHEN station_id = 'GHCND:USC00047767'
            THEN monthly_total_precipitation_in
        END) AS san_francisco_total_precipitation
    FROM vw_monthly_climate_imperial
    WHERE station_id IN ('GHCND:USW00024281', 'GHCND:USC00047767')
    GROUP BY
        year_num,
        month_num,
        month_name
)
SELECT
    year_num,
    month_num,
    month_name,
    seattle_latitude,
    san_francisco_latitude,
    seattle_avg_temperature,
    san_francisco_avg_temperature,
    ROUND(
        san_francisco_avg_temperature - seattle_avg_temperature,
        2
    ) AS san_francisco_minus_seattle_temperature,
    seattle_total_precipitation,
    san_francisco_total_precipitation,
```

```
    ROUND(
        san_francisco_total_precipitation - seattle_total_precipitation,
        2
    ) AS san_francisco_minus_seattle_precipitation,
    RANK() OVER (
        ORDER BY ABS(
            san_francisco_avg_temperature - seattle_avg_temperature
        ) DESC
    ) AS temperature_difference_rank
FROM paired_months
ORDER BY
    temperature_difference_rank,
    year_num,
    month_num;
```

Showing rows 0 - 11 (12 total, Query took 0.0076 seconds.)

```
-- OLAP Query 4: -- Compare Seattle, WA vs San Francisco, CA and rank monthly temperature gaps. WITH paired_months AS ( SELECT year_num, month_num, month_name, MAX(CASE WHEN station_id = 'GHCND:USW00024281' THEN latitude END) AS seattle_latitude, MAX(CASE WHEN station
'GHCND:USC00047767' THEN latitude END) AS san_francisco_latitude, MAX(CASE WHEN station_id = 'GHCND:USW00024281' THEN monthly_avg_temperature_f END) AS seattle_avg_temperature, MAX(CASE WHEN station_id = 'GHCND:USC00047767' THEN monthly_avg_temperature_f END) AS
san_francisco_avg_temperature, MAX(CASE WHEN station_id = 'GHCND:USW00024281' THEN monthly_total_precipitation_in END) AS seattle_total_precipitation, MAX(CASE WHEN station_id = 'GHCND:US[...]
```

+ Options

year_num	month_num	month_name	seattle_latitude	san_francisco_latitude	seattle_avg_temperature	san_francisco_avg_temperature	san_francisco_minus_seattle_temperature	seattle_total_precipitation	san_francisco_total_precipitation	san_francisco_minus_seattle_precipitation	temperature_difference_rank
2010	12	December	47.650000	37.728000	41.45	51.05	9.60	6.06	3.97	-2.09	1
2010	11	November	47.650000	37.728000	46.50	55.20	8.70	5.98	2.55	-3.43	2
2010	7	July	47.650000	37.728000	66.15	57.75	-8.40	0.85	0.01	-0.84	3
2010	2	February	47.650000	37.728000	44.26	52.55	8.29	3.52	4.10	0.58	4
2010	1	January	47.650000	37.728000	42.96	50.80	7.84	5.47	4.18	-1.29	5
2010	8	August	47.650000	37.728000	66.60	59.05	-7.55	0.94	0.05	-0.89	6
2010	3	March	47.650000	37.728000	47.55	53.25	5.70	3.85	2.63	-1.22	7
2010	6	June	47.650000	37.728000	61.99	56.65	-5.34	1.57	0.21	-1.36	8
2010	10	October	47.650000	37.728000	53.75	58.70	4.95	3.38	1.06	-2.32	9
2010	9	September	47.650000	37.728000	61.95	59.31	-2.64	1.30	0.10	-1.20	10
2010	4	April	47.650000	37.728000	51.41	53.95	2.54	2.79	1.18	-1.61	11
2010	5	May	47.650000	37.728000	57.30	55.05	-2.25	2.01	0.64	-1.37	12

Figure 5: Result of Query 4: Seattle versus San Francisco monthly difference ranking.

Query 4 compares Seattle and San Francisco month by month and ranks the months by the absolute temperature difference between the two stations. It also includes the latitude of each station and the precipitation difference for the same month. The query now computes both stations inside a single monthly comparison CTE and then ranks the resulting rows with a window function. The result is useful because it identifies the months when the climate gap between the two West Coast stations is strongest, helping connect station location with observed seasonal differences.

8.6 Query 5: Annual Climate Profile and Wettest Month by Station

```
-- OLAP Query 5:
-- Annual climate profile by station.
WITH annual_profile AS (
    SELECT
        station_id,
        station_name,
        year_num,
        ROUND(AVG(monthly_avg_temperature_f), 2) AS annual_avg_temperature,
        ROUND(MAX(monthly_max_temperature_f), 2) AS annual_max_temperature,
        ROUND(MIN(monthly_min_temperature_f), 2) AS annual_min_temperature,
```

```
        ROUND(
            MAX(monthly_max_temperature_f) - MIN(monthly_min_temperature_f),
            2
        ) AS annual_temperature_range,
        ROUND(SUM(monthly_total_precipitation_in), 2)
            AS annual_total_precipitation
    FROM vw_monthly_climate_imperial
    GROUP BY
        station_id,
        station_name,
        year_num
),
wettest_month AS (
    SELECT
        station_id,
        year_num,
        month_name AS wettest_month_name,
        monthly_total_precipitation_in AS wettest_month_precipitation,
        ROW_NUMBER() OVER (
            PARTITION BY station_id, year_num
            ORDER BY monthly_total_precipitation_in DESC, month_num
        ) AS precipitation_rank
    FROM vw_monthly_climate_imperial
)
SELECT
    ap.station_id,
    ap.station_name,
    ap.year_num,
    ap.annual_avg_temperature,
    ap.annual_max_temperature,
    ap.annual_min_temperature,
    ap.annual_temperature_range,
    ap.annual_total_precipitation,
    wm.wettest_month_name,
    wm.wettest_month_precipitation,
    RANK() OVER (
        PARTITION BY ap.year_num
        ORDER BY ap.annual_temperature_range DESC
    ) AS annual_temperature_range_rank
FROM annual_profile ap
JOIN wettest_month wm
    ON ap.station_id = wm.station_id
    AND ap.year_num = wm.year_num
WHERE wm.precipitation_rank = 1
```

```
ORDER BY
    ap.year_num,
    annual_temperature_range_rank,
    ap.station_name;
```

Showing rows 0 - 3 (4 total, Query took 0.0147 seconds.)

```
-- OLAP Query 5: -- Annual climate profile by station. WITH annual_profile AS ( SELECT station_id, station_name, year_num, ROUND(AVG(monthly_avg_temperature_f), 2) AS annual_avg_temperature, ROUND(MAX(monthly_max_temperature_f), 2) AS annual_
ROUND(MIN(monthly_min_temperature_f), 2) AS annual_min_temperature, ROUND( MAX(monthly_max_temperature_f) - MIN(monthly_min_temperature_f), 2 ) AS annual_temperature_range, ROUND(SUM(monthly_total_precipitation_in), 2) AS annual_total_precipi
BY station_id, station_name, year_num ), wettest_month AS ( SELECT station_id, year_num, month_name AS wettest_month_name, monthly_total_precipitation_in AS wettest_month_precipitation, ROW_NUMBER() OVER ( PARTITION BY stati[...]
```

+ Options

station_id	station_name	year_num	annual_avg_temperature	annual_max_temperature	annual_min_temperature	annual_temperature_range	annual_total_precipitation	wettest_month_name	wettest_month_precipitation	annual_temperature_range_rank
GHCND:USC00327027	PETERSBURG 2 N ND US	2010	38.61	80.50	-4.80	85.30	20.22	June	3.76	1
GHCND:USW00094823	PITTSBURGH INTL AP PA US	2010	51.27	82.70	20.90	61.80	38.19	June	4.30	2
GHCND:USW00024281	SEATTLE URBAN SITE WA US	2010	53.49	77.20	35.50	41.70	37.72	December	6.06	3
GHCND:USC00047767	SAN FRANCISCO OCEANSD CA US	2010	55.28	64.70	44.70	20.00	20.68	January	4.18	4

Figure 6: Result of Query 5: annual climate profile and wettest month by station.

Query 5 builds an annual climate profile for each station from the shared monthly view. It calculates annual average, maximum, and minimum temperature normals, annual temperature range, annual total precipitation, and the wettest month for each station. The result supports annual-level comparison by showing which stations have the largest temperature variability and how precipitation patterns differ across locations.

9. Limitation and Future Work

Although this project successfully implements a complete climate data warehouse pipeline, it still has several limitations.

The current dataset only includes four weather stations, so the analysis is suitable for demonstrating station-level comparison but not sufficient for drawing national-level conclusions about climate patterns across the United States.

The project mainly focuses on selected daily climate normal attributes, including minimum temperature, maximum temperature, and precipitation, which limits its ability to support broader climate analysis involving variables such as wind, humidity, snowfall, or extreme weather events.

For future work, we plan to expand the system by integrating an LLM API into the climate data warehouse. With this function, users could ask questions in natural language, and the system would automatically translate the questions into database queries, analyze the returned results, and generate readable explanations. We also plan to build a more user-friendly front page or dashboard so that users can directly interact with the database and obtain both LLM-generated insights and visualizations, such as monthly temperature trends, station comparisons, and precipitation charts. This improvement would make the project more accessible to non-technical users and turn the current backend data warehouse into an interactive climate analysis application.

10. AI Use Clarification

For this project, we made limited and appropriate use of LLM model as a form of technical assistance. Specifically, AI was used to help us refine parts of the database construction process and

revise SQL queries by offering suggestions on structure, syntax, and debugging. It served only as a supporting assistant role. All major project decisions, including schema design, data processing logic, query objectives, and the final interpretation of results, were determined, implemented, and verified by our team. Any suggestions by AI were critically evaluated and manually validated before use.